

Kata DAN CNI：面向 Kubernetes OVN 的无侵入式可扩展网络接入组件

项目项	内容
项目名称	Kata DAN CNI：面向 Kubernetes OVN 的无侵入式可扩展网络接入组件
项目导师	李亚南
申请人	林树沛
日期	2026.08.05
邮箱	linshupei2003@gmail.com

目录

Kata DAN CNI：面向 Kubernetes OVN 的无侵入式可扩展网络接入组件

- 一、项目背景
- 二、项目详细方案
 - (一) 现有代码分析以及冲突原因
 - 1. Kube-OVN 与 Kata 的传统 NetNS 路径
 - 2. Kata runtime-rs 发现 DAN 配置
 - 3. Kata runtime-rs 要求的 DAN 数据格式
 - 4. 端口竞争和资源清理冲突
 - (二) 实现方案
 - 1. DAN CNI 核心模块
 - 2. Provider 抽象框架
 - 3. Kube-OVN 只读适配器
 - 4. backend 框架模块
 - 5. host-tap backend 模块
 - 6. DAN 文件生成
- 三、开发计划

一、项目背景

Kata Containers 的 Directly Attachable Network (DAN) 允许 runtime-rs 读取以 sandbox ID 命名的 DAN JSON，将主机侧 TAP 等设备直接接入虚拟机，并在 Guest 中配置 MAC、IP 和路由。Kube-OVN 已经具备 IPAM、LSP、ACL 和 logical flow 等控制面能力，但其普通节点 CNI 会创建 veth 并使用 OVN logical port 对应的 `iface-id` 接入 `br-int`；如果 Kata DAN 再创建使用相同 `iface-id` 的 TAP，就会产生端口竞争和清理冲突。本项目计划使用 Rust 开发一个独立的 DAN CNI：普通 Pod 透明委托给原 Kube-OVN CNI，Kata Pod 则只读复用 Kube-OVN 的分配结果，创建 host TAP、接入 OVS 并生成 DAN JSON。项目不修改 Kube-OVN、OVN-Kubernetes 和 Kata runtime-rs 源码，只新增 CNI 代码、conflist、RBAC 和 DaemonSet 部署资源，也不引入额外的常驻 RPC 服务。

二、项目详细方案

（一）现有代码分析以及冲突原因

1. Kube-OVN 与 Kata 的传统 NetNS 路径

在传统路径中，容器运行时先调用 Kube-OVN CNI。Kube-OVN 为 Pod 创建 veth pair：一端放入 Pod NetNS 并命名为 `eth0`，另一端留在宿主机并加入 OVS `br-int`。这一流程对普通容器 Pod 和传统网络模式下的 Kata Pod 都可以使用。

runtime-rs 进入 Pod NetNS 后会扫描其中的网卡。当它识别到 `eth0` 是 veth 时，创建 `VethEndpoint`：

```
"veth" | "device" => {  
    let endpoint = VethEndpoint::new(...).await?;  
    ...  
}
```

`VethEndpoint::new` 继续调用 `NetworkPair::new`，在 Pod NetNS 中创建一个中间 TAP：

```
let tap_iface_name = format!("tap{idx}{TAP_SUFFIX}");  
let tap_link = create_link(handle, &tap_iface_name, queues).await?;  
...
```

这里不会发生 OVN 端口竞争。原因是 `tap0_kata` 只是虚拟机和 veth 之间的中间设备。

2. Kata runtime-rs 发现 DAN 配置

DAN 路径不再使用上述 NetNS veth 作为中间层。Kata runtime-rs 在准备 sandbox 网络资源时，会用 sandbox ID 计算 DAN 文件路径。只要这个路径存在，就优先选择 DAN，而不是普通 NetNS 网络。

```
let dan_path = dan_config_path(&config, &self.sid);  
// Network priority: DAN > NetNS  
if dan_path.exists() {  
    Some(ResourceConfig::Network(NetworkConfig::Dan(  
        DanNetworkConfig { dan_conf_path: dan_path, ... },  
    )))  
} else if let Some(netns_path) = network_env.netns.as_ref() {  
    ...  
}
```

DAN 文件放在 Kata 配置项 `runtime.dan_conf` 指向的目录下，文件名是 `<sandbox-id>.json`；

3. Kata runtime-rs 要求的 DAN 数据格式

runtime-rs 之间通过 DAN JSON 传递网络设备和 Guest 网络配置，

```
DanConfig {  
    netns: Option<String>,  
    devices: Vec<DanDevice>,  
}
```

```

DanDevice {
    name: String,
    guest_mac: String,
    device: HostTap {
        tap_name: String,
        queue_num: usize,
        queue_size: usize,
    },
    network_info: NetworkInfo {
        interface: Interface,
        routes: Vec<Route>,
        neighbors: Vec<ARPNeighbor>,
    },
}

Interface {
    ip_addresses: Vec<String>,
    mtu: u64,
    ntype: String,
    flags: u32,
}

Route {
    dest: String,
    gateway: String,
    source: String,
    scope: u32,
    flags: u32,
    mtu: u32,
}

```

MAC、IP、CIDR、网关和路由来自 Kube-OVN 已有的分配结果。runtime-rs 读取该文件后负责将 TAP 连接到虚拟机，并把 `network_info` 交给 Kata Agent 在 Guest 内配置。

4. 端口竞争和资源清理冲突

Kube-OVN 收到普通网卡的 CNI ADD 请求后，会进入 `configureNic`。这个函数先创建 veth pair，再计算 Pod 对应的 OVN `iface-id`，最后把 veth 的主机端接入 `br-int`。

```

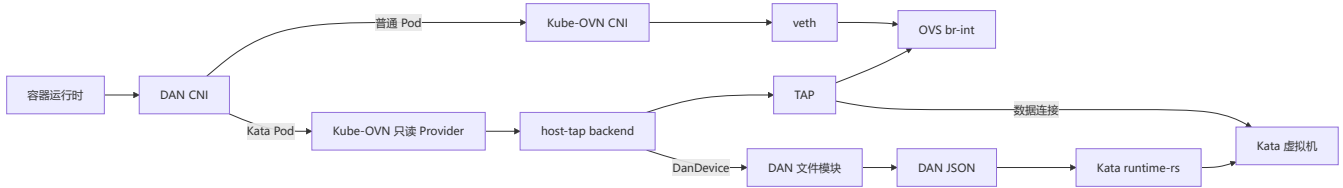
if deviceID == "" {
    hostNicName, containerNicName, err = setupVethPair(containerID, ifName, mtu)
}
...
ifaceID := ovs.PodNameToPortName(podName, podNamespace, provider)
ovs.CleanDuplicatePort(ifaceID, hostNicName)
...
args := []string{
    ovs.MayExist, "add-port", "br-int", hostNicName, "--",
    "set", "interface", hostNicName,
    "external_ids:iface-id=" + ifaceID,
    ...
}

```

veth 的主机端会成为 `br-int` 上的 OVS Interface，并通过 `external_ids:iface-id` 与控制面创建的 OVN logical port 建立对应关系。

当先执行 Kube-OVN 普通 ADD 后，veth 已经占用了目标 `iface-id`。DAN CNI 随后创建 TAP，如果它直接设置相同的 `iface-id`，本地就会有两个接口声明同一个 logical port。

(二) 实现方案



1. DAN CNI 核心模块

新组件作为 CNI 入口，接收标准 CNI 环境变量和 stdin 配置，先解析 Pod 身份，再根据持久化状态和 RuntimeClass 规则选择执行路径：

1. 普通 Pod：读取 conflist 中保存的原 Kube-OVN 配置，使用相同的 CNI 变量调用 Kube-OVN CNI，并原样返回它的最终 CNI Result。
2. Kata DAN Pod：调用只读 Provider 获取 Kube-OVN 已经完成的 MAC、IP、网关、MTU 和 logical port 信息，交给 host-tap backend 创建节点 endpoint，再生成 DAN JSON。

该模块实现三个命令：ADD 负责分流和编排；DEL 根据本地状态，对普通 Pod 调用原 Kube-OVN CNI 执行 DEL，对 Kata Pod 删除 TAP、OVS port 和 DAN 文件；CHECK 对普通 Pod 调用原 Kube-OVN CNI 执行 CHECK，对 Kata Pod 检查 DAN 设备和文件状态。

2. Provider 抽象框架

Provider 模块负责把不同网络控制面的分配结果转换成统一数据。

数据模型	主要内容
PodIdentity	Pod namespace、name、UID、node 和 RuntimeClass
NetworkSelection	provider、目标网络、主/附网络和默认路由选择
NetworkAllocation	MAC、IP/CIDR、网关、路由和 MTU
LogicalPort	logical port 名称和 <code>iface-id</code>

Provider 通过注册表加载，配置中指定 Provider 名称后，创建对应实例。同时提供 fake Provider，由它返回固定的四类数据，并跑通 ADD、CHECK 和 DEL。

3. Kube-OVN 只读适配器

Kube-OVN 只读适配器是 Provider 框架的一个具体实现。它根据 CNI 传入的 Pod namespace、name、UID、网卡名和 provider 找到本次网络请求，并校验 Pod UID，避免 Pod 同名重建后读错数据。

适配器主要从 Pod annotations 中读取 Kube-OVN 已经分配的 MAC、IP/CIDR、网关、路由、逻辑交换机和分配状态。如果 Kube-OVN Controller 还没有完成分配，适配器会在超时时间内等待。数据就绪后，它按当前 Kube-OVN 版本的端口命名规则得到 logical port 名称和 `iface-id`，然后整理成 `PodIdentity`、`NetworkSelection`、`NetworkAllocation` 和 `LogicalPort` 四类统一数据，交给后续 backend 使用。

整个过程只需要 Kubernetes API 的 `get/list/watch` 权限，不修改 Pod annotations、IP/Subnet CRD 或 OVN 控制面数据。

4. backend 框架模块

数据模型	主要内容
<code>NetworkContext</code>	Provider 数据、sandbox ID、ifname、netns 和 backend 配置
<code>BackendState</code>	实际创建的设备、OVS bridge/port、 <code>iface-id</code>
<code>DanDevice</code>	Guest 接口、主机设备参数以及 IP、MTU、路由等网络信息

每个 backend 实现统一的创建、检查和删除接口，并通过以下机制接入 DAN CNI：

- backend 注册：**框架在进程内维护注册表，host-tap 作为内置实现注册。出现重名或未知 backend 时直接报错。
- 配置解析：**框架先解析 backend 名称、默认 backend、provider/网络匹配规则等公共配置，再把设备相关的私有参数交给对应 backend 解析。
- 能力校验：**每个 backend 声明自己支持的设备类型、IPv4/IPv6、multi-queue、OVS 接入和 CHECK 等能力。框架根据 `NetworkContext` 生成本次请求需要的能力集合，若所选 backend 不满足要求，则在创建 TAP 或 OVS port 前返回错误；
- backend 选择：**ADD 根据 `NetworkSelection` 和配置中的 provider/网络匹配规则选择 backend。

5. host-tap backend 模块

host-tap backend 是框架的一个具体实现，backend 处理过程为：

- 根据 sandbox ID、网络和 ifname 生成稳定的 TAP 名称，在主机网络命名空间中创建 TAP；
- 设置 TAP 的 MTU、链路状态和队列参数，Guest MAC 使用 Kube-OVN Provider 给出的分配结果；
- 检查 `br-int` 以及目标 `iface-id`。如果同一 `iface-id` 已被普通 veth 或其他组件占用，则报告冲突；
- 将 TAP 加入 `br-int`，并在对应 OVS Interface 上设置 `iface-id`、sandbox ID、Pod UID 和 owner 等 external IDs；
- 返回 `BackendState` 和 `DanDevice`，分别供 DEL/CHECK 和 DAN 文件生成使用。

6. DAN 文件生成

DAN JSON 是新 CNI 与 runtime-rs 之间的交接文件，也是 runtime-rs 选择 DAN 网络的依据。

`DanConfig.devices` 中的单个元素对应 `DanDevice`。

```
pub(crate) struct DanDevice {
    name: String,
    guest_mac: String,
    device: Device,
    network_info: NetworkInfo,
}
```

DAN 文件模块把 `DanDevice` 组织成目标 Kata 版本要求的 `DanConfig`，文件名使用 `<sandbox-id>.json`。写入时先创建临时文件，完成序列化和校验后再原子替换正式文件。文件发布成功后，组件返回标准 CNI Result。

三、开发计划

阶段	周期	主要工作
学习与调研	第 1—4 周	学习 Kata DAN、Kube-OVN、OVN、CNI，阅读相关源码
功能实现	第 5—9 周	实现 DAN CNI 的 ADD、DEL 和 CHECK，完成 Provider/backend 框架、Kube-OVN 只读适配器、host-tap backend、DAN 文件生成及相关测试
文档编写	第 10—12 周	完善代码和部署使用、测试报告和项目总结等文档